

[Download All Experiments \(ZIP\)](#) [Download CC-lab.txt](#) [Download IEEE Paper.docx](#)

[Google Drive Link](#) [Toofaan Lab Website](#)

Exp-1 — EC2 Access

a) Connect using PuTTY (.ppk)

1. Create an EC2 with a key pair of .ppk, copy the IP of it.
2. Open PuTTY, paste the IP.
3. Left side: Connection → SSH → Auth → Credentials, browse the .ppk and click Open.
4. If Linux login as: **ec2-user**, if Ubuntu login as: **ubuntu**. Test: `whoami` (console)

b) Convert .pem to .ppk

Convert .pem file to .ppk using PuTTYGen, save as private key file, then follow the same process as above.

c) RDP (Windows EC2)

1. Create an EC2 — in security groups add RDP in the custom TCP, select OS: Windows.
2. Click on Connect → using RDP client → click Get Password → upload the .pem file to decrypt.
3. Click the Windows symbol and search RDP, paste the IP of the EC2.
4. Login as **Administrator**, password: decrypted password.
5. You can see the new system.

Exp-2 — Volumes & Snapshots

Volumes

a) Attach a Volume

1. Create an EC2, select an AZ.
2. Create a volume with the same AZ, size anything.
3. Attach volume to the running EC2, device name: `/dev/sdf`.
4. Open EC2 console and run `lsblk`.

b) Format and Mount the Volume

After attaching and connecting to EC2:

```
sudo su
lsblk -fs
fdisk /dev/nvme1n1    (m, n, p, , , , w)
lsblk
mkfs.xfs /dev/nvme1n1p1
mkdir /mnt/vish
mount /dev/nvme1n1p1 /mnt/vish
df -h
blkid /dev/nvme1n1p1
(COPY UUID)
nano /etc/fstab
(inside that, last line add) =UUID=<UUID-value>    /mnt/vish    xfs    defaults,nofail
0    0
mount -a
reboot and check for files
```

c) Move Volume to Another EC2

1. Create another EC2 with the same AZ.
2. Detach the volume from the old EC2 and attach to the new one.
3. Connect to the new EC2 and run:

```
sudo su
mkdir /mnt/vish
mount /dev/nvme1n1p1 /mnt/vish
lsblk -fs
```

Snapshots

1. For a given volume you created, click on Actions → Create Snapshot.

2. From the Snapshots section, click on Copy Snapshot and select some region.
 3. Switch to that region and create a volume for it from Actions.
 4. Attach an EC2 in that region to the created volume with the same AZ. (run `lsblk` if needed)
-

Exp-3 — EFS (Elastic File System)

1. Create 2 EC2s with 2 different availability zones and include NFS in the custom TCP.
2. Create a new security group (NFS) and attach it to the EFS
3. Create an EFS with only 2 availability zones.
4. Open consoles of both EC2s.
5. Run the following commands on both:

```
sudo su
mkdir efs
yum install -y amazon-efs-utils
(connect the efs with dns) efs → connect → dns → paste in console
cd efs
in any 1 do: nano h.txt and put text
(it shows in the other EC2)
```

EXP-4 — S3

a) Public Access to S3 Object

1. Create an S3 bucket, allow ACLs under ownership.
2. Block all public access — uncheck that (very important).
3. Enable versioning (better, because others need it).
4. After creation: Bucket → Permissions → ACL → Public Access → right side tick.
5. Upload any image, click on it → Permissions → ACL → Public Access → left and right tick.
6. Properties → Copy link → open in incognito.

b) Versioning

1. Enable versioning (compulsory).
2. Upload a text file, change it locally, and upload again.
3. Buckets → Objects → click Show Versions → you can see both versions.
4. Delete the latest version and the old version still shows up.

c) Cross Region Replication

1. Create 1 more bucket in a different region with the same process including versioning enabled.
2. Go to first bucket → Management → Replication Rules.
3. Status = Enable, Source bucket = Apply to all objects, Destination bucket = select the created one, IAM Role = Lab Role. Replicate existing objects → No (this option comes after creating).
4. Upload any file in bucket 1 — it will automatically show in the 2nd bucket.

d) Static Website Hosting

1. Upload `index.html` and `error.html` into S3 following the normal process including ACL settings and allowing static hosting. Put index and error filenames in the fields.
2. Click on `index.html` → Permissions → Everyone → both tick. Same with `error.html`.
3. Bucket → Properties → scroll to Static Website Hosting → copy URL → check.

EXP-5 — VPC

1. Create a VPC, CIDR: `10.0.0.0/16`.
2. Create subnets:
 - **Public:** AZ-east-1a, subnet CIDR: `10.0.1.0/24`
 - **Private:** AZ-east-1a, subnet CIDR: `10.0.2.0/24`
3. Create an Internet Gateway, attach the VPC (shown after creation).
4. Create Route Tables:
 - **Public:** Create → Actions → Edit Routes → Add Route → Destination: `0.0.0.0/0`, Internet Gateway (custom made) → Actions → Edit Subnet Associations → add the public subnet.
 - **Private:** Create → Actions → Edit Subnet Associations → add the private subnet.

5. Public subnet → Actions → Edit Subnet Settings → Auto-assign IP (tick).
6. Create a security group, add the custom VPC under creation, inbound rules: SSH, HTTP, HTTPS.
7. Create an EC2 for the public subnet: Edit Network → VPC (custom), Subnet: public, select existing security group: custom made.
8. Do the same for the EC2 of the private subnet.
9. Connecting to both: public subnet EC2 will connect, private will not.

Exp-6 — Bastion Server

1. Follow the same process as Exp-5 until creation of the route tables.
2. Create an Elastic IP.
3. Create a NAT Gateway: Method of Elastic IP → Manual, select the created IP.
4. Attach the NAT Gateway to the private route table.
5. Create 2 EC2 instances for public and private subnets and attach the 2 subnets.
6. Open the public EC2 instance and connect it via SSH. In the terminal where .pem file is located, run:

```
ssh -i "key.pem" ec2-user@<public-ip>
```

7. Open another terminal and copy the key to the public EC2:

```
scp -i yourkey.pem yourkey.pem ec2-user@<public-ip>:~
```

8. In terminal 1 (connected to public EC2), run:

```
chmod 400 yourkey.pem
```

9. Open the EC2 with private subnet and connect it via SSH command from terminal 1:

```
ssh -i yourkey.pem ec2-user@<private-ip>
```

10. After connection success, run to check internet access:

```
ping google.com
```

Exp-7 — S3-DynamoDB Integration

1. Create an S3 bucket normally.
2. Create a DynamoDB table with a partition key as **id**.
3. Create a Lambda function: Author from scratch, Role: **LabRole**, Runtime: **Python**.
4. Add trigger to Lambda: Select S3 in trigger config, select the bucket.
5. In the Lambda code, put this:

```
import boto3
from uuid import uuid4

def lambda_handler(event, context):
    s3 = boto3.client("s3")
    dynamodb = boto3.resource('dynamodb')

    # Check if the 'Records' key is present in the event
    if 'Records' in event:
        # Iterate over each record in the event
        for record in event['Records']:
            bucket_name = record['s3']['bucket']['name']
            object_key = record['s3']['object']['key']
            size = record['s3']['object'].get('size', -1)
            event_name = record.get('eventName', 'Unknown')
            event_time = record.get('eventTime', 'Unknown')

            dynamoTable = dynamodb.Table('table-name')
            dynamoTable.put_item(
                Item={
                    'id': str(uuid4()),
                    'Bucket': bucket_name,
                    'Object': object_key,
                    'Size': size,
                    'Event': event_name,
                    'EventTime': event_time
                })
    else:
```

```
# Log a message indicating that the 'Records' key is missing
print("No 'Records' key found in the event.")
```

6. Click **Deploy**.
7. Upload something in the S3 bucket, then click **Explore Items** in DynamoDB, click **Run**.
8. In Lambda, show logs if asked.

Exp-8 — SNS & SQS

Part 1: SNS Email Notification

1. Create SNS with any name, Type: **Standard**.
2. Create subscription: Protocol: **Email**, Endpoint: your email.
3. Confirm subscription from email.
4. Publish message: Put something in subject and body, then send.

Part 2: S3 to SNS Integration

1. Create S3 bucket normally.
2. Edit access policy of SNS and put this:

```
{
  "Version": "2012-10-17",
  "Id": "example-ID",
  "Statement": [
    {
      "Sid": "Example SNS topic policy",
      "Effect": "Allow",
      "Principal": {
        "Service": "s3.amazonaws.com"
      },
      "Action": [
        "SNS:Publish"
      ],
      "Resource": "SNS topic ARN",
      "Condition": {
```

```

    "ArnLike": {
      "aws:SourceArn": "s3 bucket ARN"
    },
    "StringEquals": {
      "aws:SourceAccount": "account id"
    }
  }
}
]
}

```

3. Open S3 → Properties → Event Notification → Create Event Notification.
4. Event name: any name, Event type: **All objects create events**, Destination: **SNS topic**.
5. Test by uploading something, check email.

Part 3: SQS Basic Operations

1. Create SQS, Type: **Standard**, any name.
2. Click **Send and Receive Messages**, message body: some message, click **Send**.
3. Poll messages to see the message, you can delete also.

Part 4: S3 → SNS → SQS → Lambda Pipeline

1. Create S3, SNS, SQS normally.
2. Create subscription in SNS: Protocol: **SQS**, Endpoint: **SQS ARN**.
3. Add access policy in SQS by editing and putting:

```

{
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "sns.amazonaws.com"
      },
      "Action": "sqs:SendMessage",
      "Resource": "SQS ARN",
      "Condition": {
        "ArnEquals": {
          "aws:SourceArn": "SNS ARN"
        }
      }
    }
  ]
}

```



```

    }
  }
}
]
}

```

4. Add access policy in SNS also:

```

{
  "Version": "2012-10-17",
  "Id": "example-ID",
  "Statement": [
    {
      "Sid": "Example SNS topic policy",
      "Effect": "Allow",
      "Principal": {
        "Service": "s3.amazonaws.com"
      },
      "Action": [
        "SNS:Publish"
      ],
      "Resource": "SNS topic ARN",
      "Condition": {
        "ArnLike": {
          "aws:SourceArn": "S3 bucket ARN"
        },
        "StringEquals": {
          "aws:SourceAccount": "Account ID"
        }
      }
    }
  ]
}

```

5. Open S3 → Properties → Event Notification → Event name: **S3UploadEvent**, Event type: **All object create events**, Destination: **SNS topic**.

6. Test by uploading something in S3, poll messages from SQS.

7. Configure Lambda: Create Lambda function normally, add trigger: **SQS**, select your SQS.

8. Lambda code:

```
def lambda_handler(event, context):
    for record in event['Records']:
        print("Message received from SQS:")
        print(record['body'])

    return {
        'statusCode': 200,
        'body': 'Message processed successfully'
    }
```

9. Deploy, upload something in S3, and this will trigger everything. Watch logs in Lambda.

Exp-9 — ELB + Auto Scaling

Part 1: EC2 + Load Balancer

1) Create EC2 Instances (2 servers)

Launch instance → Amazon Linux → t2.micro

Security group:

- SSH (22) → your IP
- HTTP (80) → anywhere

Run in both:

```
sudo su
yum update -y
yum install httpd -y
systemctl start httpd
systemctl enable httpd
```

Server 1:

```
echo "This is Server 1" > /var/www/html/index.html
```

Server 2:

```
echo "This is Server 2" > /var/www/html/index.html
```

Result: 2 working web servers

2) Create Load Balancer

1. Security group: EC2 → Security Groups → Create Security Group, allow HTTP, HTTPS traffic.
2. Create Load Balancer: EC2 → Load Balancers → Create Load Balancer → **Application Load Balancer** (everything default), select all AZ, attach the above security group.
3. Create new target group: Put name, other default.
4. Register targets: Attach the instances.
5. Add the target group to load balancer.

3) Test

- Copy DNS of LB
- Open in browser
- Refresh → alternates servers
- Output: traffic distributed

Part 2: Auto Scaling

1) Create AMI

- EC2 → Select instance
- Actions → Image → Create Image
- Name: anything
- This is your base template

2) Create Launch Template

EC2 → Launch Templates → Create

Config:

- AMI → select created AMI
- Instance type → t2.micro
- Key pair → select
- Security group → allow HTTP

3) Create Auto Scaling Group

EC2 → Auto Scaling Groups → Create

Config:

- Launch template → select
- VPC + subnets → select all

4) Attach Load Balancer

Select:

- Existing Load Balancer
- Target group

Health check:

- Type → HTTP
- Grace period → 300 sec

5) Set Capacity

- Desired: 2
- Min: 1
- Max: 4

6) Create ASG

Review → Create

Part 3: Testing

1) Check Instances

- Target group → healthy instances
- ASG → instances running

2) Test Auto Healing

- Terminate one instance
- Result: ASG launches new instance automatically

3) Test Load Balancing

- Open LB DNS
- Refresh

Exp-10 — Elastic Beanstalk

1. Create Elastic Beanstalk, some name.
2. Create environment → Web server environment.
3. Platform: **Tomcat**.
4. Upload WAR code from: <https://tomcat.apache.org/tomcat-6.0-doc/appdev/sample/>
5. Role: **LabRole**, Key pair: select.
6. Configure network: VPC (default), Subnets, Enable public IP.
7. Instance: **t2.micro**, Type: **Load Balancer**, Min: 1, Max: 2.
8. Create.
9. Open the URL (domain link).
10. Upload and Deploy → Upload same with different version label, and open URL again.

Exp-11 — Amazon Lex

1. Open Lex → Create bot → Name (everything default).
 2. Intent → Some name → Add utterances:
 - I want to book a hotel
 - Book a room
 - Reserve hotel
 - I need a room
 3. Create slots:
 - **age** → number → question
 - Advanced → Branch → if {age} < 18 → message: not eligible
 - More slots: **city**, **date**, **nights**
 4. Custom slot:
 - Slot types → Add slot type → Add blank slot type
 - **RoomType**: single, double, suite
 5. Add slot **RoomType**, Advanced → Slot prompts → More prompt options → Add → Add card groups → button.
 6. Initial Response:

Welcome to Hotel Booking! What is your name?
 7. Confirmation:

Do you want to confirm booking in {location} for {nights} nights?
 8. Build → Test.
-

Exp-12 — Creating IAM User with Permissions

1. IAM → IAM users → Create → Name → Provide user access (tick).
2. Give credentials, Access: **AmazonS3FullAccess**.
3. Login, try creating EC2 (will not happen), create bucket.
4. IAM → Users → Security credentials → Create access key → CLI.
5. In CMD on your system:

```
aws configure
```

Enter:

- Access Key ID
- Secret Access Key
- Default region: (e.g., ap-south-1)
- Default output: json

Run commands:

```
aws s3 ls  
aws s3 mb s3://your-unique-bucket-name  
aws iam list-users    (won't work)
```

Exp-13 — IAM Roles

1. IAM Console → Roles → Create role.
2. Use case: Choose **EC2**.
3. Permissions: **AmazonS3FullAccess**.
4. Name (something), Create.
5. Create EC2 instance normally.
6. Actions → Security → Modify IAM role.
7. Select **EC2-S3-Role** → Update.
8. Connect to EC2 and run:

```
aws s3 ls  
aws ec2 describe-instances    (doesn't work)
```